

SignIn-UI-Frontend

Complete Project Learning & Architecture Guide

A Deep, File-by-File, Flow-by-Flow Engineering Manual for the Stackly Auth & Multi-Step Enterprise Workspace Onboarding Platform

PROJECT CODEBASE

SignIn-UI-Frontend (stackly-auth)

ARCHITECTURE STYLE

50/50 Split Auth Layout with React Context Multi-Step Pipeline

CORE TECH STACK

React 19.2 + TypeScript + Vite 8.3 + Tailwind v4 + MUI v9

TARGET AUDIENCE

Engineering Leads, POC Reviewers, Frontend Developers

TOC Table of Contents

This guide is organized into 40 comprehensive chapters covering every file, prop, state, hook, and architecture decision in the project.

1. Executive Overview & Problem Solved	01	2. High-Level Architecture & 50/50 Layout	02
3. Complete Folder & File Map	03	4. Application Startup Flow (Browser to DOM)	04
5. Package.json & Dependencies Table	05	6. Routing Architecture (React Router v7)	06
7. Page-by-Page Deep Dive (All 11 Pages)	07	8. Component Architecture & Hierarchy	08
9. Props Flow & Project Prop Catalog	09	10. State Management (Context + Local)	10
11. Form Architecture & Controlled Inputs	11	12. Form Validation Engine (validation.ts)	12
13. Event Handling & Interactive Handlers	13	14. TypeScript Implementation & Interfaces	14
15. Styling Architecture: Tailwind CSS v4	15	16. Material UI (MUI v9) Integration & Synergy	16
17. CSS Architecture & Custom Directives	17	18. Left Panel: Visual Branding Architecture	18
19. Right Panel: Dynamic Form Container	19	20. Asset Catalog & Media Pipeline	20
21. End-to-End Cross-Page Data Flow	21	22. Module Interconnection & Dependency Graph	22
23. End-to-End User Flow & Screen Transitions	23	24. Button-by-Button Interactive Reference	24
25. Conditional Rendering Strategies	25	26. Responsive Breakpoint Behavior	26
27. Error Handling, Validation & Lockout Logic	27	28. Backend Integration Status & Mock Architecture	28
29. Authentication & Security Simulation	29	30. React Concepts Master Checklist	30
31. TypeScript Concepts Master Checklist	31	32. Line-by-Line Code Walkthroughs	32
33. POC & Interview Speaking Script with Q&A	33	34. "How I Built This" Development Narrative	34
35. Architecture Rationale: "Why This Way?"	35	36. Technical Improvement Opportunities	36
37. Complete System Architecture Diagrams	37	38. Technical Project Glossary	38
39. Final Project Cheat Sheet	39	40. Self-Assessment & Mastery Checklist	40

01 Project Overview & Problem Solved

1.1 What is SignIn-UI-Frontend?

SignIn-UI-Frontend (internal package name: `stackly-auth`) is an enterprise-grade, multi-stage authentication and workspace onboarding frontend application built for the **One Enterprise** cloud platform by **Stackly**.

In modern B2B SaaS applications, onboarding an enterprise client is fundamentally different from a consumer sign-up. A consumer only needs an email and password. An enterprise onboarding process requires collecting organization metadata (name, slug/code, industry, employee head-count, regional compliance jurisdiction, and logo branding), configuring the initial root Super Administrator identity, confirming legal compliance and data governance policies, and provisioning a dedicated tenant workspace subdomain (e.g., `https://acmecorp.oneenterprise.io`).

1.2 The Core Problem it Solves

Enterprise software onboarding is traditionally clunky, overwhelming, and fragmented. If users face a 25-field monolithic form on a single page, bounce rates spike, and users abandon onboarding. **SignIn-UI-Frontend** solves this with:

Cognitive Load Reduction: The onboarding journey is decomposed into three clear, progressive milestones:

1. **Step 1: Organization Details** — Establishing company identity, workspace slug, and geographic settings.
2. **Step 2: Super Admin Account** — Creating the privileged administrator identity with enforced enterprise password complexity.
3. **Step 3: Review & Confirm** — Verifying legal acceptance, authorization, and data processing agreements before tenant creation.

High-Confidence Visual Branding: A sticky, 50% split-screen desktop panel presents enterprise trust badges (SOC 2 Type II, ISO 27001, 99.95% SLA) and an interactive visual orbital diagram illustrating how *People*, *Process*, and *Progress* interconnect through One Enterprise (1E).

Dual-Track Authentication Flow: It seamlessly integrates two major user journeys:

- **Existing Tenant Sign-In:** Workspace resolution (`acmecorp.oneenterprise.io`) with Work Email validation and SSO (Google, Microsoft, SAML).
- **New Organization Provisioning:** The complete 3-step registration wizard transferring captured metadata across steps into a unified state store.

1.3 Conversational Explanation: How the App Works

CONVERSATIONAL EXPLANATION STYLE

"When a user opens the application, it immediately directs them to `/signin`. On large screens, the screen splits cleanly down the middle: the left side displays our interactive visual branding panel with our mountain backdrop, orbital cards, and security assurances. The right side contains the interactive form.

If an existing employee types their workspace and email, they proceed into the authentication pipeline. If a company is registering for the first time, they click 'Create an account'. This opens our step-by-step onboarding wizard. In Step 1, they enter their company name—our code automatically suggests a clean uppercase code like 'ABC-TECH'. They select their industry, country, and state. When they click 'Continue', that data isn't lost; our React Context stores it. Step 2 asks for their Admin details. Step 3 asks them to accept legal agreements. Once accepted, they click 'Create account' and reach the Welcome screen with their assigned workspace URL!"

02 High-Level Architecture & 50/50 Layout

2.1 The 50/50 Desktop Split Layout Pattern

The visual and architectural foundation of `SignIn-UI-Frontend` is the **50/50 Split Screen Architecture** defined inside `src/components/AuthLayout.tsx`. This layout separates the viewport into two distinct horizontal partitions on screens at or above the Tailwind `lg` breakpoint (1024px):

```
+-----+
| Viewport: 100vw x 100vh (h-screen w-full overflow-hidden bg-white) |
+-----+
| LEFT PANEL (50% on lg:block, hidden on mobile) | RIGHT PANEL (100% mobile, 50% on lg:w-1/2) | | |
| Component:                                     | Component: rendered inside |
| |                                               | |
| | - Fixed viewport height (h-full)              | | - Independent vertical scrolling (overflow-y) |
| | - Locked non-scrollable stage (overflow-hidden) | | - Scrollbar hidden via custom .no-scrollbar |
| | - Full mountain background image               | | - Centered form column (max-w-[456px] - [560px]) |
| | - Stackly Brand Logo                          | | - StepProgress (3 segments indicator) |
| | - Orbital Diagram (1E Center + 3 Nodes)        | | - Dynamic Page Injection via React Router |
| | - Interactive Tabs (Secure, Scalable, Future)   | | (/signin, /organization, /admin, /review, |
| | - Compliance Footer (SOC 2, ISO 27001, SLA)     | | /welcome, /login, /2fa, /locked, etc.) |
+-----+
```

2.2 Why This Architectural Separation Matters

Zero Layout Shift: Because the left branding panel is mounted once by `AuthLayout` as a persistent route layout, navigating between `/signin`, `/organization`, `/admin`, and `/review` changes only the right-hand `<Outlet />`. The left panel does NOT unmount, re-render, or flicker.

Independent Scroll State: The left panel is pinned to `max-h-screen overflow-hidden`. The right panel has `overflow-y-auto` with `.no-scrollbar`. When a user scrolls down a long form like `OrganizationForm` (which has 8 input groups and a file upload), the branding and trust assurances stay perfectly locked in the user's field of view.

Graceful Mobile Degradation: On viewports below 1024px (tablets and smartphones), the left panel receives `hidden`, allowing the right-side form container to take 100% of the screen width with natural mobile scrolling and comfortable touch targets.

03 Complete Project Folder & File Structure

The following tree represents the exact, actual filesystem structure of `SignIn-UI-Frontend`. No imaginary files or directories are included.

```
SignIn-UI-Frontend/
├── public/
│   ├── favicon.svg
│   ├── icons.svg
│   ├── mountains.png
│   ├── stackly-logo.png
│   └── stackly-logoo.png
│
├── src/
│   ├── assets/
│   │   ├── hero.png
│   │   ├── react.svg
│   │   └── vite.svg
│   │
│   ├── components/
│   │   ├── auth/
│   │   │   ├── AgreementCheckbox.tsx
│   │   │   ├── DividerText.tsx
│   │   │   ├── FileUpload.tsx
│   │   │   ├── FormField.tsx
│   │   │   ├── PrimaryButton.tsx
│   │   │   ├── SocialLoginButton.tsx
│   │   │   └── StepProgress.tsx
│   │   │
│   │   ├── organization/
│   │   │   └── OrganizationForm.tsx
│   │   │
│   │   ├── ui/
│   │   │   └── Button.tsx
│   │   │
│   │   ├── AgreementCheckbox.tsx
│   │   ├── AuthLayout.tsx
│   │   ├── ProgressSteps.tsx
│   │   └── VisualBrandingPanel.tsx
│   │
│   ├── context/
│   │   ├── OnboardingContext.tsx
│   │   ├── OnboardingContextValue.ts
│   │   └── useOnboarding.ts
│   │
│   ├── pages/
│   │   ├── AccountLocked.tsx
│   │   ├── AdminAccountPage.tsx
│   │   ├── CheckEmail.tsx
│   │   ├── EnterPassword.tsx
│   │   ├── ForgotPassword.tsx
│   │   ├── OrganizationDetailsPage.tsx
│   │   ├── ReviewConfirm.tsx
│   │   ├── ReviewConfirmPage.tsx
│   │   ├── SignInPage.tsx
│   │   ├── Success.tsx
│   │   ├── TwoFactor.tsx
│   │   ├── Welcome.tsx
│   │   └── WelcomePage.tsx
│   │
│   ├── types/
│   │   └── onboarding.ts
│   │
│   ├── utils/
│   │   └── validation.ts
│   │
│   ├── App.css
│   ├── App.tsx
│   ├── index.css
│   └── main.tsx
│
├── .gitignore
├── index.html
├── package.json
└── package-lock.json
```

Static public assets served directly by Vite at root
Browser tab favicon icon
SVG symbol sprite sheet
High-res scenic mountain image for left branding panel
Primary Stackly brand logo
Alternate branded logo asset

Application source code
Bundled assets processed through Vite module graph
Hero illustration graphic
Standard React logo
Standard Vite logo

Reusable UI component library
Domain-specific authentication & form primitives
Custom styled checkbox with optional badge
Horizontal rule with centered text ("or continue with")
Drag-and-drop file upload with preview and validation
Universal input & dropdown wrapper with error states
Dark navy action button (#0f1330) with hover states
Google, Microsoft, and SAML SSO authentication buttons
3-segment progress indicator with gradient states

Organization domain components
Comprehensive Step 1 onboarding form (439 lines)

Material UI styled component wrappers
MUI Button styled as PrimaryButton and SecondaryButton

Re-export bridge to ./auth/AgreementCheckbox
50/50 Split Screen Master Layout wrapper with
Re-export wrapper around StepProgress
Pixel-perfect Left Branding Panel (277 lines)

Global React State Management
Provider component storing organization & admin state
TypeScript interface & Context object creation
Custom hook with guard check for accessing context

Routed Page Views (11 distinct page components)
Security lockout screen triggered after 5 failed logins
Step 2: Super Admin Account creation form
Password reset link dispatch notification page
Password entry page with attempt counter and lockout trigger
Self-service work email password recovery form
Step 1: Organization details page container
Step 3: Terms, authorization, and data processing review
Re-export alias for ReviewConfirm
Step 1 Identify: Workspace & Work Email sign-in page
Post-authentication success confirmation card
6-digit OTP authenticator verification screen
Final Step 6: Workspace provisioned & email verify screen
Re-export alias for Welcome

Global TypeScript Type Definitions
Interfaces for OrganizationDetails, AdminAccount, State

Helper utilities & Business Logic
Validation functions (email, workspace, password, org-code)

Legacy / starter CSS rules
Master application component with Router and Routes
Tailwind v4 import, @theme variables, and font imports
React 19 bootstrap entry point (createRoot + StrictMode)

Git file exclusions (node_modules, dist, etc.)
Single-Page Application HTML entry template
Project manifest, dependencies, and execution scripts
Deterministic dependency lockfile

```

└─ tsconfig.app.json      # TypeScript compiler configuration for client code
└─ tsconfig.json         # TypeScript root solution configuration
└─ tsconfig.node.json    # TypeScript compiler configuration for Vite scripts
└─ vite.config.ts        # Vite bundler configuration with React and Tailwind plugins

```

3.1 File-by-File Breakdown: Purpose, Imports & Exports

File Path	What it Imports	What it Exports	Purpose & What Breaks if Removed
src/main.tsx	StrictMode , createRoot , ./index.css , App	None (entry file)	Bootstraps React 19 into the #root DOM element. If removed, the entire app fails to run; blank white screen.
src/App.tsx	react-router-dom , OnboardingProvider , AuthLayout , all 11 pages	App (default)	Defines all URL route mappings, sets up OnboardingProvider context, and establishes route fallback. If removed, routing collapses.
src/components/AuthLayout.tsx	Outlet , VisualBrandingPanel	AuthLayout (named)	Provides the two-column 50/50 responsive split layout. Houses the persistent left branding panel and dynamic right outlet. If removed, layout collapses.
src/components/VisualBrandingPanel.tsx	React, MUI Icons (Security , Lock , AccessTime)	VisualBrandingPanel (named)	Renders the left-side branding UI with background image, orbital diagram, 1E core, interactive tabs, and compliance badges.
src/context/OnboardingContext.tsx	React, OnboardingContextValue , types/onboarding	OnboardingProvider (named)	Holds global state for organization and admin across steps. Provides updateOrganization , updateAdmin , and resetOnboarding .
src/context/useOnboarding.ts	useContext , OnboardingContextValue	useOnboarding (named)	Custom hook providing safe access to OnboardingContext. Throws an explicit error if consumed outside the provider.
src/utils/validation.ts	None (pure TypeScript)	validateRequired , validateEmail , validateWorkspace , validatePassword , validateConfirmPassword , generateOrgCode	Validation engine and slug auto-generator. If removed, all form submissions break without validation logic.

04 Application Startup Flow (Browser to DOM)

4.1 What Happens When You Run `npm run dev`?

When a developer executes `npm run dev` in the terminal, the following sequence occurs under the hood:

```
Step 1: Terminal executes 'npm run dev' → maps to 'vite' binary in package.json
|
Step 2: Vite initializes development server at http://localhost:5173
|   - Loads vite.config.ts
|   - Registers @vitejs/plugin-react (fast HMR transform)
|   - Registers @tailwindcss/vite (compiles Tailwind v4 stylesheet in-memory)
|
Step 3: Browser requests http://localhost:5173/
|   - Server serves index.html
|   - Browser parses
|
|   - Browser finds script:
|
Step 4: Vite transpiles & serves src/main.tsx via native ES Modules (ESM)
|   - main.tsx imports './index.css' (injects compiled Tailwind rules & Google Fonts)
|   - main.tsx imports App from './App.tsx'
|   - Calls createRoot(document.getElementById('root')).render(...)
|
Step 5: React 19 creates Root Fiber and mounts
|
Step 6: App.tsx executes in memory:
|   - Instantiates <HTML5 History API>
|   - Instantiates <initializes organization & admin state>
|   - Evaluates against current window.location.pathname
|
Step 7: Route matching and rendering:
|   - If URL is "/" → Matches < />
|     Browser URL changes to /signin without page reload
|   - Route /signin matches nested route under < />
|   - renders:
|     Left:
|     Right:
|   → which resolves to
|
Step 8: Final Paint: User sees the complete 50/50 Sign In Page in ~120 milliseconds!
```

4.2 The Role of StrictMode

In `src/main.tsx`, `<App />` is wrapped in `<StrictMode>`. In development mode, React intentionally invokes component render functions and state updater functions twice. This is an intentional debugging tool to help developers detect side-effects, memory leaks, and unsafe mutations before shipping to production. In production builds (`npm run build`), `StrictMode` adds zero overhead.

05 Package.json & Dependencies Table

5.1 Actual Project Dependencies Breakdown

Inspecting the actual `package.json` reveals a modern, cutting-edge frontend toolchain utilizing React 19, Tailwind CSS v4, and Material UI v9.

Package Name	Version	Type	Where Used	Technical Purpose & Necessity
<code>react</code>	<code>^19.2.8</code>	Runtime	Entire Project	The core component library providing JSX syntax, component lifecycle, React Hooks (<code>useState</code> , <code>useCallback</code> , <code>useRef</code> , <code>useEffect</code>).
<code>react-dom</code>	<code>^19.2.8</code>	Runtime	<code>src/main.tsx</code>	The browser rendering glue for React. Provides <code>createRoot</code> to mount the React virtual DOM tree into HTML5.
<code>react-router-dom</code>	<code>^7.18.4</code>	Runtime	<code>App.tsx</code> , all Pages	Client-side single-page routing without page reloads. Provides <code>BrowserRouter</code> , <code>Routes</code> , <code>Route</code> , <code>Navigate</code> , <code>Outlet</code> , and <code>useNavigate</code> .
<code>tailwindcss</code>	<code>^4.3.3</code>	Runtime/Dev	<code>src/index.css</code>	Tailwind CSS version 4 engine. Delivers utility-first styling with zero-runtime CSS generation and CSS variable theming.
<code>@tailwindcss/vite</code>	<code>^4.3.3</code>	Runtime/Plugin	<code>vite.config.ts</code>	Official Vite plugin for Tailwind v4. Compiles utility classes directly inside Vite's lightning-fast build pipeline without PostCSS.
<code>@mui/material</code>	<code>^9.4.0</code>	Runtime	<code>components/ui/Button.tsx</code> , <code>EnterPassword.tsx</code>	Material UI v9 component library. Supplies enterprise input elements like <code>TextField</code> , <code>Checkbox</code> , <code>FormControlLabel</code> , and <code>styled</code> .
<code>@mui/icons-material</code>	<code>^9.4.0</code>	Runtime	<code>VisualBrandingPanel.tsx</code> , <code>AdminAccountPage.tsx</code>	MUI SVG icons: <code>SecurityOutlined</code> , <code>LockOutlined</code> , <code>AccessTimeOutlined</code> , <code>ShieldOutlined</code> , <code>VisibilityOutlined</code> .
<code>@emotion/react</code> & <code>@emotion/styled</code>	<code>^11.14.0</code>	Runtime	MUI dependency	Underlying CSS-in-JS styling engines required by Material UI to process dynamic component styling and <code>styled()</code> wrappers.
<code>lucide-react</code>	<code>^1.47.0</code>	Runtime	<code>FormField.tsx</code> , <code>FileUpload.tsx</code> , <code>SocialLoginButton.tsx</code>	Feather-style modern lightweight SVG icons: <code>ChevronDown</code> , <code>Upload</code> , <code>X</code> , <code>Users</code> , <code>Eye</code> , <code>EyeOff</code> , <code>Lock</code> , <code>Check</code> .
<code>vite</code>	<code>^8.3.0</code>	Dev	Build system	Next-generation frontend dev server and bundler. Provides Hot Module Replacement (HMR) and optimized Rollup production builds.
<code>typescript</code>	<code>~6.0.2</code>	Dev	Type checking	Static type system providing compile-time type validation, autocompletion,

Package Name	Version	Type	Where Used	Technical Purpose & Necessity
				and refactoring safety across all <code>.ts</code> and <code>.tsx</code> files.
<code>eslint</code>	<code>^8.57.0</code>	Dev	Code Quality	Rust-based, ultra-fast linter that analyzes JavaScript/TypeScript code 50x-100x faster than traditional ESLint.

5.2 Dependencies vs devDependencies

dependencies are packages necessary to execute the application in the user's browser runtime (React, React Router, Tailwind, MUI, Lucide). **devDependencies** are development-only tools required to transpile, type-check, bundle, or lint code during the development lifecycle (Vite, TypeScript, eslint, type definition files). They are not bundled into the final production payload.

06 Complete Routing Architecture (React Router v7)

6.1 Route Table & Layout Mapping

All application routing is declared declaratively in `src/App.tsx` using `react-router-dom` (v7.18.4). Notice how `<AuthLayout />` acts as a parent layout route wrapping every authenticated screen.

Path	Rendered Component	Layout Wrapper	Flow Purpose
<code>/</code>	<code><Navigate to="/signin" replace /></code>	None	Root redirect: forwards users straight to the sign-in screen.
<code>/signin</code>	<code><SignInPage /></code>	<code>AuthLayout</code>	Step 1 Identify: Workspace and work email entry point.
<code>/organization</code>	<code><OrganizationDetailsPage /></code>	<code>AuthLayout</code>	Step 1 of 3: Organization Details form.
<code>/organization/details</code>	<code><OrganizationDetailsPage /></code>	<code>AuthLayout</code>	Alias route for Step 1 Organization Details.
<code>/admin</code>	<code><AdminAccountPage /></code>	<code>AuthLayout</code>	Step 2 of 3: Super Administrator account configuration.
<code>/create-admin</code>	<code><AdminAccountPage /></code>	<code>AuthLayout</code>	Alias route for Step 2 Admin Account.
<code>/review</code>	<code><ReviewConfirm /></code>	<code>AuthLayout</code>	Step 3 of 3: Review & Confirm agreements.
<code>/review-confirm</code>	<code><ReviewConfirm /></code>	<code>AuthLayout</code>	Alias route for Step 3 Review & Confirm.
<code>/welcome</code>	<code><Welcome /></code>	<code>AuthLayout</code>	Final onboarding completion & workspace ready screen.
<code>/login</code>	<code><EnterPassword /></code>	<code>AuthLayout > CenteredAuthWrapper</code>	Password entry screen for returning users.
<code>/2fa</code>	<code><TwoFactor /></code>	<code>AuthLayout > CenteredAuthWrapper</code>	6-digit authenticator code verification.
<code>/forgot-password</code>	<code><ForgotPassword /></code>	<code>AuthLayout > CenteredAuthWrapper</code>	Password recovery request screen.
<code>/check-email</code>	<code><CheckEmail /></code>	<code>AuthLayout > CenteredAuthWrapper</code>	Password recovery email dispatch confirmation.
<code>/locked</code>	<code><AccountLocked /></code>	<code>AuthLayout > CenteredAuthWrapper</code>	Security lockout page after 5 failed password attempts.
<code>/success</code>	<code><Success /></code>	<code>AuthLayout > CenteredAuthWrapper</code>	Login authentication success indicator.
<code>*</code>	<code><Navigate to="/signin" replace /></code>	None	Catch-all wildcard fallback route: sends unknown URLs to <code>/signin</code> .

6.2 The CenteredAuthWrapper Component

Inside `src/App.tsx` lines 21–27, a dedicated helper component wraps secondary authentication screens:

```
const CenteredAuthWrapper: React.FC<{ children: React.ReactNode }> = ({ children }) => (  
  <div className="w-full min-w-0 flex-1 flex flex-col justify-center items-center p-8 lg:p-16">
```

```
<div className="w-full max-w-md">
  {children}
</div>
</div>
);
```

Why was this written? The primary onboarding pages (`OrganizationForm` , `AdminAccountPage` , `ReviewConfirm`) require a wider container (`max-w-[560px]`) aligned to the top with precise padding (`pt-[104px]`) to accommodate step progress bars. Secondary auth pages (like entering a password, 2FA, or lockout) are shorter and look best centered vertically in the viewport inside a compact `max-w-md` (448px) column.

07 Page-by-Page Deep Dive (All 11 Pages)

`SignIn-UI-Frontend` implements two complete workflows: the **Primary Onboarding Flow** (a 5-step wizard) and the **Secondary Authentication Flow** (a 6-step login & recovery suite).

7.1 Primary Page 1: Sign In Page (`src/pages/SignInPage.tsx`)

Page Metadata & Routing

Route: `/signin` (also redirected from root `/` and fallback `*`)

Visual Milestone: STEP 1 OF 3 • IDENTIFY

Primary Layout Container: `max-w-[456px]` with desktop top-padding `lg:pt-[146px]`

Purpose: Identifies returning enterprise tenants and routes new users into organization registration.

Components Used: `PrimaryButton`, `DividerText`, `SocialLoginButton`, `FormField`.

State:

`workspace` (string, default: `'acmecorp'`) — Company workspace slug.

`email` (string, default: `''`) — Corporate email address.

`errors` (object: `{ workspace?: string; email?: string }`) — Validation error strings.

`touched` (object: `{ workspace?: boolean; email?: boolean }`) — Dirty tracking flags.

`showFindModal` (boolean, default: `false`) — Toggles the inline helper card explaining how to find a workspace.

Special UI Pattern: Split Workspace Input: The workspace input field joins an editable text input with a fixed right suffix box displaying `.oneenterprise.io` in a muted mono font. This enforces subdomain mental mapping.

What Happens on Submit:

1. Validates `workspace` against alphanumeric + hyphen regex.
2. Validates `email` against standard email format.
3. If valid, calls `updateOrganization({ organizationCode: workspace.toUpperCase() })` in Context so the user's workspace choice auto-seeds Step 1!
4. Executes `navigate('/organization')`.

7.2 Primary Page 2: Organization Details (`OrganizationDetailsPage.tsx` & `OrganizationForm.tsx`)

Page Metadata & Routing

Route: `/organization` and `/organization/details`

Visual Milestone: STEP 1 OF 3 • ORGANIZATION DETAILS

Primary Layout Container: `max-w-[560px]` with top padding `lg:pt-[104px]`

Purpose: Captures all enterprise organization profile attributes: name, code/slug, type, industry, employee size, country, state, city, time zone, and company logo.

Components Used: `StepProgress` (step 1 active), `FormField` (text and select variants), `FileUpload`, `PrimaryButton`.

Dependent Dropdowns: The State/Province dropdown dynamically binds to the selected Country using the internal lookup dictionary `statesByCountry` (supports India, United States, United Kingdom, Singapore, Germany, Australia). Changing country immediately re-seeds the available state options!

Slug Auto-Generation: When typing in "Organization Name" (e.g., "Acme International Systems"), the helper function `generateOrgCode()` automatically computes "ACME-INTE" in uppercase. Users can click "edit manually" to override it.

What Happens on Continue: All form values are submitted to `updateOrganization(...)` in `OnboardingContext`, and the browser navigates to `/admin`.

7.3 Primary Page 3: Super Admin Account (`src/pages/AdminAccountPage.tsx`)

Page Metadata & Routing

Route: `/admin` and `/create-admin`
Visual Milestone: `STEP 2 OF 3 · SUPER ADMIN ACCOUNT`
Notice Banner: Blue badge explaining automatic assignment of the `SUPER_ADMIN` role.

Purpose: Configures the initial root administrator credentials: First Name, Last Name, Official Email, Mobile Number, Username, Password, and Confirm Password.

Dynamic Headline: "This is the account you'll use to manage `{organizationName}`." (organizationName is reactively retrieved from `OnboardingContext`).

Password Visibility Toggles: Independent boolean states (`showPassword`, `showConfirmPassword`) toggle input types between "password" and "text" with MUI icons (`VisibilityOutlined` / `VisibilityOffOutlined`).

Strict Password Validation: Enforces minimum 8 characters, at least one digit (`/[0-9]/`), and at least one special symbol. Confirms that password and confirmPassword match.

What Happens on Continue: Calls `updateAdmin(values)` in Context and navigates to `/review`.

7.4 Primary Page 4: Review & Confirm (`src/pages/ReviewConfirm.tsx`)

Page Metadata & Routing

Route: `/review` and `/review-confirm`
Visual Milestone: `STEP 3 OF 3 · TERMS & AUTHORIZATION`
Action Guard: 'Create account' button is strictly disabled until 3 mandatory checkboxes are accepted.

Purpose: Legal compliance gateway. Users must review and explicitly accept legal agreements before the tenant workspace is created.

Agreements Card: Uses custom `AgreementCheckbox` components:

1. Terms of Service & Privacy Policy (Mandatory)
2. Confirmation of Authority to bind `{organizationName}` and accept Super Admin liability (Mandatory)
3. Data Processing Agreement (DPA) regarding GDPR/enterprise storage (Mandatory)
4. Product updates & security email notices (Optional — marked with uppercase OPTIONAL badge)

Button Enabling Gate: `const canCreateAccount = termsAccepted && authorizationAccepted && dataProcessingAccepted;`

What Happens on Create Account: Navigates to `/welcome`.

7.5 Primary Page 5: Welcome / Success (`src/pages/Welcome.tsx`)

Page Metadata & Routing

Route: `/welcome`
Status Indicator: Circular green badge with emerald checkmark (`bg-[#e9f8ef]` `text-[#2ca35c]`).

Purpose: Confirms successful workspace provisioning and notifies the administrator that an activation link has been dispatched to their official email.

Computed Workspace Domain: Computes the live tenant URL:

```
const rawCode = organization.organizationCode?.trim() || 'ABC-TECH';
const workspaceDomain = rawCode.toLowerCase().replace(/^[a-z0-9-]/g, '') + '.oneenterprise.io';
```

Resend Verification Simulation: Clicking 'Resend verification' sets `resent = true`, rendering an emerald status badge: `✓`. Verification email resent to `{admin.officialEmail}!`, which auto-resets after 4 seconds via `setTimeout`.

Return to Sign In: Clicking 'Go to sign in' navigates to `/signin`.

7.6 Secondary Authentication Flow Pages

Page File	Route	Step Indicator	Special Technical Features
<code>EnterPassword.tsx</code>	<code>/login</code>	<code>STEP 2 OF 3 · PASSWORD</code>	Demo password check (<code>'password123'</code>), failed attempt counter, triggers automatic lockout navigation to <code>/Locked</code> upon 5 failed attempts. Includes remember device checkbox and user switcher card.

Page File	Route	Step Indicator	Special Technical Features
TwoFactor.tsx	/2fa	STEP 3 OF 3 · VERIFY	6 individual input boxes with auto-focus forward advancing via <code>useRef</code> array and backspace backward jump. Expiration timer display. Navigates to <code>/success</code> .
ForgotPassword.tsx	/forgot-password	PASSWORD RECOVERY	Work email input field with Material UI <code>TextField</code> . Navigates to <code>/check-email</code> upon clicking 'Send reset link'.
CheckEmail.tsx	/check-email	Notification View	Shows mail icon badge, expiration notice (30 minutes), 'Resend email' secondary button, and 'Back to sign in' button.
AccountLocked.tsx	/locked	Security Warning	Red lockout badge, 15-minute countdown card (Try again in 15:00), quick reset password link, audit log notice.
Success.tsx	/success	Auth Complete	Emerald circular checkmark icon with subtitle: "Redirecting to your dashboard..."

08 Component Architecture & Hierarchy

8.1 Master Component Hierarchy Tree

The following diagram illustrates how components are nested at runtime:



8.2 Reusable Atomic Components Breakdown

Component	File Path	Key Props	Architectural Functionality
StepProgress	components/auth/StepProgress.tsx	currentStep , totalSteps , className	Renders the horizontal 3-segment progress indicator with active navy fill, completed blue gradients, and upcoming gray bars.
FormField	components/auth/FormField.tsx	id , label , required , error , isSelect , options , value , onChange	Dual-mode form control. If isSelect=true , renders a styled dropdown with custom ChevronDown icon. Otherwise, renders an HTML5 text/email/password input with red validation labels.
FileUpload	components/auth/FileUpload.tsx	value , previewUrl , onChange , optional	Full drag-and-drop file upload zone with HTML5 file validation (PNG/JPG only, 5MB limit),

Component	File Path	Key Props	Architectural Functionality
			image preview thumbnail, and remove action.
PrimaryButton	components/auth/PrimaryButton.tsx	children , heightClass , disabled , type	Standardized dark navy (#0f1330) primary action button with focus rings and disabled styling.
AgreementCheckbox	components/auth/AgreementCheckbox.tsx	checked , onChange , optional , children	Custom accessible checkbox with keyboard navigation (Space/Enter toggle) and optional metadata tag badge.
SocialLoginButton	components/auth/SocialLoginButton.tsx	provider ('google' 'microsoft' 'sso'), onClick	Branded SSO button with authentic Google 4-color SVG, Microsoft 4-box SVG, or SAML SSO icon.
DividerText	components/auth/DividerText.tsx	text , className	Subtle gray divider line with centered background-matched badge text ("or continue with").

09 Props Flow & Real Project Props Catalog

9.1 Real Component Props in Action

In React, **props** (short for properties) represent read-only inputs passed from a parent component down to a child component. Props make components reusable and predictable.

Example 1: StepProgress Props in `AdminAccountPage.tsx`

```
// In AdminAccountPage.tsx (Line 73):  
<StepProgress currentStep={2} totalSteps={3} />
```

Origin: Hardcoded integer literal `2` supplied by `AdminAccountPage` because it represents Step 2 of the onboarding flow.

Consumer: `StepProgress.tsx`.

Runtime Effect: `StepProgress` loops from 1 to 3. Segment 1 (index 0) gets the completed gradient `bg-gradient-to-r from-[#76a4f0] to-[#8781f0]`; segment 2 (index 1) gets the active solid navy `bg-[#0e1333]`; segment 3 (index 2) gets the inactive gray `bg-[#e7eae6]`.

Example 2: FormField Polymorphic Props in `OrganizationForm.tsx`

```
// In OrganizationForm.tsx (Lines 267-283):  
<FormField  
  id="org-type"  
  label="Organization Type"  
  required  
  isSelect  
  value={orgType}  
  onChange={(e) => setOrgType(e.target.value)}  
  options={[  
    { value: 'Enterprise', label: 'Enterprise' },  
    { value: 'Mid-Market', label: 'Mid-Market' },  
    { value: 'Small Business', label: 'Small Business' }  
  ]}  
>
```

Origin: State variable `orgType` and state setter `setOrgType` owned by `OrganizationForm`.

Consumer: `FormField.tsx`.

Runtime Effect: Because `isSelect={true}`, `FormField` renders an HTML `<select>` rather than an `<input>`, maps through `options` to render `<option>` children, and positions a `ChevronDown` icon in the right margin.

11 Form Architecture & Controlled Inputs

11.1 Controlled Inputs Explained

Every input field in this project is a **Controlled Component**. In React, a controlled component is an input element whose value is driven by React state, not internal DOM state:

```
<input
  id="org-name"
  type="text"
  value={orgName}           // 1. Single Source of Truth
  onChange={handleOrgNameChange} // 2. Intercepts Keystrokes
/>
```

How it works at runtime:

1. User presses key 'A'.
2. Browser fires `onChange` event.
3. `handleOrgNameChange` extracts `e.target.value`.
4. Handler calls `setOrgName(val)`.
5. React schedules a re-render.
6. React passes the new `orgName` string to the input's `value` prop.
7. The letter 'A' renders on screen.

11.2 File Upload & Drag-and-Drop Architecture

`src/components/auth/FileUpload.tsx` provides a production-grade file upload pipeline:

Hidden File Input: A real HTML `<input type="file" className="hidden">` is referenced via `const inputRef = useRef<HTMLInputElement>(null)`.

Click Delegation: Clicking the styled dashed container executes `inputRef.current?.click()`.

HTML5 Drag Events: Handles `onDragOver`, `onDragLeave`, and `onDrop` with `e.preventDefault()` to prevent the browser from opening the image in a new tab.

Object URL Previews: When a file is selected, it generates an in-memory blob preview via `URL.createObjectURL(file)` for immediate thumbnail display.

12 Form Validation Engine (validation.ts)

12.1 The Validation Functions in src/utils/validation.ts

All business validation rules are encapsulated in pure, unit-testable TypeScript functions:

Function	Validation Criteria	Error Message Returned
<code>validateRequired(val, name)</code>	Checks if string is empty or only whitespace (<code>!value !value.trim()</code>).	<code>"{fieldName} is required"</code>
<code>validateEmail(email, name)</code>	1. Checks required. 2. Evaluates regex: <code>/^[^\s@]+@[^\s@]+\.[^\s@]+\$/</code> .	<code>"Please enter a valid email address"</code>
<code>validateWorkspace(slug)</code>	1. Checks required. 2. Evaluates regex: <code>/^[a-zA-Z0-9-]+\$</code> (letters, digits, hyphens only).	<code>"Workspace can only contain letters, numbers, and hyphens"</code>
<code>validatePassword(pwd)</code>	1. Checks required. 2. Length ≥ 8 . 3. At least one digit: <code>/[0-9]/</code> . 4. At least one symbol: <code>/[!@#\$\$%^&*(),.?":{} <>]/</code> .	<code>"Password must be at least 8 characters" / "...must include at least one number" / "...one symbol"</code>
<code>validateConfirmPassword(p1, p2)</code>	1. Checks confirm required. 2. Compares <code>p1 !== p2</code> .	<code>"Passwords do not match"</code>
<code>generateOrgCode(name)</code>	Sanitizes special characters, splits into words, takes the first 4 letters of word 1 and first 4 of word 2 with a hyphen. E.g., "Google Cloud" -> "GOOG-CLOU".	Returns generated slug string.

12.2 Real-time "Touched" Dirty Tracking

To ensure an exceptional user experience, error messages do NOT flash when a user first lands on an empty form. Each field tracks a `touched[fieldName]` boolean flag. Errors only display after the user leaves the field (`onBlur`) or attempts to submit the form!

13 Event Handling & Interactive Handlers

13.1 Key Event Handlers in the Project

React employs a Synthetic Event system that wraps native browser events for cross-browser consistency. The project implements five key event patterns:

Event Name	Where Implemented	Handler Function	Runtime Execution Sequence
<code>onSubmit</code>	<code>OrganizationForm.tsx</code> , <code>SignInPage.tsx</code> , <code>AdminAccountPage.tsx</code>	<code>handleSubmit(e)</code>	1. Calls <code>e.preventDefault()</code> to halt browser reload. 2. Executes <code>validateAll()</code>. 3. If valid, dispatches context updates. 4. Navigates to next route.
<code>onChange</code>	<code>FormField.tsx</code> , <code>OrganizationForm.tsx</code>	<code>handleOrgNameChange(e)</code>	1. Extracts <code>e.target.value</code>. 2. Updates local form state. 3. Computes <code>generateOrgCode()</code> if slug is in auto-mode. 4. Clears active error if touched.
<code>onBlur</code>	All Input Fields	Inline arrow functions	Fires when input loses focus. Sets <code>touched[field] = true</code> and immediately checks validation to give instant feedback.
<code>onKeyDown</code>	<code>AgreementCheckbox.tsx</code> , <code>TwoFactor.tsx</code>	<code>handleKeyDown(e)</code>	In <code>AgreementCheckbox</code> : intercepts Space/Enter to toggle check state for accessibility. In <code>TwoFactor</code> : intercepts Backspace to shift focus to previous OTP box.
<code>onDrop</code> & <code>onDragOver</code>	<code>FileUpload.tsx</code>	<code>onDrop(e)</code> , <code>onDragOver(e)</code>	1. <code>e.preventDefault()</code> prevents browser file opening. 2. Extracts <code>e.dataTransfer.files[0]</code>. 3. Validates file type and <= 5MB size. 4. Creates blob URL preview.

14 TypeScript Implementation & Type Safety

14.1 Core Type Interfaces in `src/types/onboarding.ts`

TypeScript guarantees complete contract enforcement across the application. The central models are defined cleanly:

```
export interface OrganizationDetails {
  organizationName: string;
  organizationCode: string;
  organizationType: string;
  industry: string;
  companySize: string;
  country: string;
  state: string;
  city: string;
  timeZone: string;
  organizationLogo?: File | null;
  logoPreviewUrl?: string | null;
}

export interface AdminAccount {
  firstName: string;
  lastName: string;
  officialEmail: string;
  mobileNumber: string;
  username: string;
  password: string;
  confirmPassword: string;
}

export interface OnboardingState {
  organization: OrganizationDetails;
  admin: AdminAccount;
}
```

14.2 Partial Updates via Generic Types

In `src/context/OnboardingContextValue.ts`, updater functions utilize TypeScript's built-in utility type `Partial<T>`:

```
export interface OnboardingContextType {
  organization: OrganizationDetails;
  admin: AdminAccount;
  updateOrganization: (data: Partial<OrganizationDetails>) => void;
  updateAdmin: (data: Partial<AdminAccount>) => void;
  resetOnboarding: () => void;
}
```

Why this matters: `Partial<OrganizationDetails>` allows callers to submit only the fields they want to change (for instance, when `SignInPage` updates only `{ organizationCode: workspace.toUpperCase() }`), without having to pass the other 10 properties.

15 Styling Architecture: Tailwind CSS v4 in Depth

15.1 The Modern Tailwind v4 Engine

This project is built on the brand new **Tailwind CSS v4** (`@tailwindcss/vite ^4.3.3`). Unlike Tailwind v3, Tailwind v4:

Does NOT require a `tailwind.config.js` or `postcss.config.js` file!

Uses the native Vite plugin `@tailwindcss/vite` configured in `vite.config.ts`.

Directly imports into CSS via `@import "tailwindcss";` in `src/index.css`.

Configures custom theme tokens directly inside CSS using the official `@theme` directive!

15.2 Breakdown of Real Tailwind Classes Used in the Project

Tailwind Class Snippet	Where Used	CSS Property & Design Purpose
<code>hidden lg:block lg:w-1/2</code>	<code>AuthLayout.tsx</code>	Hides the left panel on screens under 1024px. Displays it at 50% width on desktop.
<code>h-screen overflow-hidden</code>	<code>AuthLayout.tsx</code>	Locks total viewport height to 100vh and prevents outer browser scrollbars from appearing.
<code>overflow-y-auto no-scrollbar</code>	<code>AuthLayout.tsx</code> <code><main></code>	Enables smooth vertical scrolling on the right form container while hiding ugly browser scrollbars.
<code>rounded-[28px] bg-white/90 backdrop-blur-xl</code>	<code>VisualBrandingPanel.tsx</code> (IE Card)	Creates the ultra-modern glassmorphic frosted-glass card with 28px border-radius and blur filter.
<code>bg-gradient-to-r from-[#76a4f0] to-[#8781f0]</code>	<code>StepProgress.tsx</code>	Creates the polished blue-to-indigo linear gradient fill for completed steps in the progress indicator.
<code>bg-[#0f1330] hover:bg-[#18204a] text-white</code>	<code>PrimaryButton.tsx</code>	The primary brand color: deep midnight navy with smooth hover shift to indigo navy.

16 Material UI (MUI v9) Integration & Synergy

16.1 How MUI and Tailwind Coexist

A common question in enterprise interviews: *"Why does your project use both Tailwind CSS and Material UI?"*

THE ENGINEERING SEPARATION OF CONCERNS

Tailwind CSS v4 governs all macro-layouts, page containers, responsive grids, orbital diagrams, and custom branding cards.

Material UI v9 (`@mui/material` & `@mui/icons-material`) provides specialized accessible enterprise controls and battle-tested icon vectors.

16.2 Real Project Examples of MUI Usage

Custom Styled Components (`src/components/ui/Button.tsx`):

```
import { Button as MuiButton } from '@mui/material';
import { styled } from '@mui/material/styles';

export const PrimaryButton = styled(MuiButton)({
  backgroundColor: '#0F172A',
  color: 'ffffff',
  textTransform: 'none',
  fontWeight: 600,
  fontSize: '14px',
  padding: '12px 24px',
  borderRadius: '8px',
  '&:hover': { backgroundColor: '#1E293B' }
});
```

Material UI Icons: Used in `VisualBrandingPanel.tsx` (`SecurityOutlinedIcon` , `LockOutlinedIcon` , `AccessTimeOutlinedIcon`) and `AdminAccountPage.tsx` (`ShieldOutlinedIcon` , `VisibilityOutlinedIcon` , `VisibilityOffOutlinedIcon`).

Enterprise Password Field: In `EnterPassword.tsx` , MUI's `TextField` with `InputAdornment` and `IconButton` hosts the password reveal toggle with clean accessible focus rings.

17 CSS Architecture & Custom Directives

17.1 Analysis of `src/index.css`

`src/index.css` is concise, modern, and acts as the central styling manifest:

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&family=JetBrainsMono:wght@400;500;600;700&displaysubset=latin');
@import "tailwindcss";

@theme {
  --color-brand-dark: #0F172A;
  --color-brand-blue: #4F46E5;
  --color-brand-lightblue: #F0F4F8;
  --font-sans: 'Inter', system-ui, sans-serif;
  --font-mono: 'JetBrains Mono', monospace;
}

body {
  font-family: 'Inter', sans-serif;
  -webkit-font-smoothing: antialiased;
}

/* Hide scrollbar while preserving natural scroll behavior */
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  -ms-overflow-style: none;
  scrollbar-width: none;
}
```

Google Fonts: Loads *Inter* for standard clean body typography and *JetBrains Mono* for technical labels, code, and step indicators.

@theme Directive: Exposes CSS custom properties that integrate with Tailwind v4's class generator (e.g. `text-brand-dark`, `font-mono`).

.no-scrollbar Utility: Strips scrollbar visual elements across WebKit (Chrome, Edge, Safari), Firefox (`scrollbar-width: none`), and IE/legacy Edge (`-ms-overflow-style: none`) while maintaining scrollability.

18 Left Panel: Visual Branding Architecture

18.1 Deep Inspection of `VisualBrandingPanel.tsx`

The left branding panel is a showcase of sophisticated CSS positioning, SVG vector geometry, and glassmorphism. It contains four vertical layers:

```
+-----+
| 1. BACKGROUND LAYERS (Z-0) |
| - Full coverage background image: url('/mountains.png') (bg-cover bg-center) |
| - Soft white tint overlay: bg-white/25 (ensures high contrast text readability) |
+-----+
| 2. TOP HEADER (Z-20) |
| - Brand Logo:  STACKLY |
+-----+
| 3. HERO & ORBITAL SYSTEM (Z-10) |
| Left Side Text: |
| - Category tracker: "CLOUD PLATFORM · HRMS · CRM · ERP · FINANCE · AI" |
| - Bold H1: "One identity. Infinite Potential." (with blue highlight) |
| - Subtitle: "A unified platform to connect your people, data and operations" |
| |
| Right Side Orbital System (290px x 330px SVG canvas): |
| - SVG Curved Paths (Cubic Bezier curve vectors stroke="#60a5fa") |
| - Center Core: Frosted Glass 1E Card (108px x 108px, cyan/sky glow, 1E text) |
| - Node 1: "People / Together" card with user group icon (top-right) |
| - Node 2: "Process / Simpler" card with stacked layers icon (bottom-left) |
| - Node 3: "Progress / Faster" card with bar chart icon (bottom-right) |
+-----+
| 4. BOTTOM TABS & COMPLIANCE FOOTER (Z-20) |
| - Interactive tab triggers: [Secure] [Scalable] [Future-Ready] |
| (Active tab gets 2.5px blue indicator pill bar) |
| - Slogan: "BUILT FOR A BRIGHTER TOMORROW" |
| - Security Badges: SOC 2 Type II · ISO 27001 · 99.95% uptime SLA |
+-----+
```

19 Right Panel: Dynamic Form Container

19.1 Layout & Sizing Standards

The right panel inside `src/components/AuthLayout.tsx` is configured with:

```
<main className="w-full lg:w-1/2 h-full overflow-y-auto overflow-x-hidden no-scrollbar flex flex-col items-center justify-start bg-white">
  <Outlet />
</main>
```

19.2 The StepProgress Indicator Standard

Every onboarding screen renders the `<StepProgress />` bar at the top of the form. The bar has a fixed maximum width of `560px` and height of `4px`, split into 3 segments separated by `6px` gaps.

Completed Segments: Render with `bg-gradient-to-r from-[#76a4f0] to-[#8781f0]`.

Active Current Segment: Renders with solid midnight navy `bg-[#0e1333]`.

Upcoming Inactive Segments: Render with soft light gray `bg-[#e7eae6]`.

20 Asset Catalog & Media Pipeline

20.1 Media Catalog Table

Asset Filename	Location	Format	How it is Used	Visual Role
mountains.png	public/	PNG Image	CSS background in <code>VisualBrandingPanel.tsx</code> via <code>url('/mountains.png')</code>	Full-bleed cinematic mountain range backdrop for the entire left panel.
stackly-logo.png	public/	PNG Image	HTML <code></code> tag in <code>VisualBrandingPanel.tsx</code>	Primary corporate header logo.
stackly-logoo.png	public/	PNG Image	Public static asset	High-resolution brand logo asset variant.
hero.png	src/assets/	PNG Image	Vite bundled asset	Alternative graphic illustration asset.
favicon.svg	public/	SVG Vector	Linked in <code>index.html</code> (<code><link rel="icon"></code>)	Browser tab identity icon.

21 End-to-End Cross-Page Data Flow

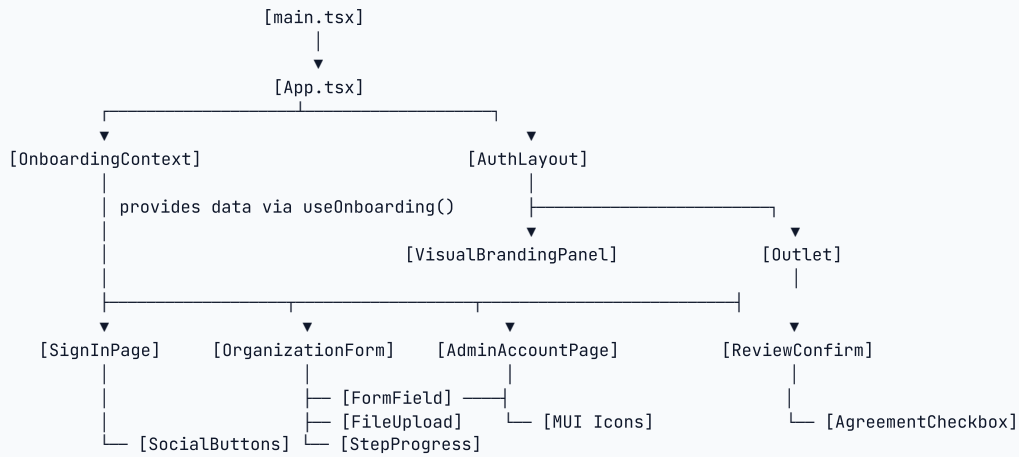
21.1 Tracing Data Movement Through the Pipeline

A critical concept to explain to an engineering manager is how data moves between distinct page routes without getting wiped out:

```
[Screen 1: /signin]
  User enters workspace: "acmecorp" and email: "you@acmecorp.com"
  Click "Continue"
  Calls: updateOrganization({ organizationCode: "ACMECORP" })
  |
  ▼ React Context (OnboardingContext) stores organizationCode = "ACMECORP"
[Screen 2: /organization]
  Component mounts. Reads organizationCode from Context.
  User fills Organization Name: "Acme Corporation"
  Auto-slug generates code, or user keeps "ACMECORP".
  User fills Industry, Country ("United States"), State ("California"), City ("San Francisco").
  User uploads Logo ("acme-logo.png").
  Click "Continue"
  Calls: updateOrganization({ organizationName, organizationCode, industry, country, state, city, ... })
  |
  ▼ React Context updates entire organization object in memory
[Screen 3: /admin]
  Component mounts.
  Reads organizationName: "Acme Corporation" → dynamically injects into headline!
  User enters First Name ("Ananya"), Last Name ("Rao"), Email ("ananya@acmecorp.com"), Password.
  Click "Continue"
  Calls: updateAdmin({ firstName, lastName, officialEmail, mobileNumber, username, password })
  |
  ▼ React Context updates admin object in memory
[Screen 4: /review]
  Component mounts.
  Reads organization.organizationName → displays in confirmation authorization text!
  User checks 3 mandatory legal agreements.
  Click "Create account"
  |
  ▼ Final step reached
[Screen 5: /welcome]
  Component mounts.
  Reads organization.organizationCode ("ACMECORP") → computes "acmecorp.oneenterprise.io".
  Reads admin.officialEmail ("ananya@acmecorp.com") → displays verification notice!
  User clicks "Go to sign in" → Returns to /signin.
```

22 Module Interconnection & Dependency Graph

22.1 How Modules Connect Across the Codebase



23 End-to-End User Flow & Screen Transitions

23.1 The 6-Stage User Journey

1. **Stage 1: Tenant Discovery** (`/signin`) — User opens the app. Enters workspace name and work email. If new, clicks 'Create an account'.
2. **Stage 2: Workspace Setup** (`/organization`) — User completes the company profile. Slug is auto-generated in real time. Regional states populate dynamically based on country selection.
3. **Stage 3: Privilege Elevation** (`/admin`) — User establishes the root Super Administrator. Strict password complexity rules (length, number, symbol) are enforced before proceeding.
4. **Stage 4: Compliance Attestation** (`/review`) — User reviews all obligations, binds their organization, and accepts terms of service and DPA.
5. **Stage 5: Activation & Provisioning** (`/welcome`) — User views their assigned workspace subdomain and receives instructions to verify their email.
6. **Stage 6: Day-One Sign-In** (`/login` & `/2fa`) — Returning administrator signs in with password, passes two-factor authenticator verification, and lands on `/success`.

24 Button-by-Button Interactive Reference

Button Label	Screen / Location	Visual Appearance	Fired Event & Function	State Changes & Navigation Outcome
Continue	SignInPage.tsx	Navy pill (h-[44px] bg-[#0f1330])	handleSubmit(e)	Validates inputs. Saves workspace slug to Context. Navigates to /organization .
Google / Microsoft / SSO	SignInPage.tsx	White bordered with provider SVG icon	onClick={() => navigate('/organization')}	Simulates external OAuth/SSO login and routes user into workspace configuration.
Find it here	SignInPage.tsx	Inline blue link (text-[#232a5c])	onClick={() => setShowFindModal(!showFindModal)}	Toggles showFindModal to render inline dismissible workspace helper card.
Continue	OrganizationForm.tsx	Large navy button (h-[48px] bg-[#0f1330])	handleSubmit(e)	Validates all 4 required fields. Commits company profile to Context. Navigates to /admin .
◀ Back	AdminAccountPage.tsx	Muted gray text with chevron	onClick={() => navigate('/organization')}	Navigates backward to Step 1 without losing already entered data.
Password Eye Icon	AdminAccountPage.tsx	MUI Visibility SVG icon	onClick={() => setShowPassword(!showPassword)}	Flips showPassword boolean, toggling input between masked password and clear text .
Continue	AdminAccountPage.tsx	Large navy button (h-[48px] bg-[#0f1330])	handleSubmit(e)	Enforces password complexity. Commits admin profile to Context. Navigates to /review .
Create account	ReviewConfirm.tsx	Disabled light gray (bg-[#e9edf5]) or Active navy	onClick={handleCreateAccount}	Guarded by canCreateAccount . When all 3 mandatory boxes are checked, navigates to /welcome .
Go to sign in	Welcome.tsx	Large navy button (h-[48px] bg-[#0f1330])	onClick={handleSignIn}	Navigates user back to /signin to begin day-one login.
Resend verification	Welcome.tsx	Blue underlined text link	onClick={handleResend}	Sets resent = true , displaying emerald confirmation notice for 4000ms.

25 Conditional Rendering Strategies

25.1 Real Conditional Patterns in the Codebase

Short-Circuit Evaluation (`&&`):

```
// In ReviewConfirm.tsx: renders OPTIONAL tag only if optional is true
{optional && (
  <span className="text-[10px] font-mono font-medium text-[#9aa3b2] tracking-[0.1em] uppercase ml-1.5">
    OPTIONAL
  </span>
)}
```

Ternary Operator (`? :`):

```
// In FormField.tsx: dynamically renders HTML <select> vs <input>
{isSelect ? (
  <select ...>...</select>
) : (
  <input ... />
)}
```

File Preview vs Dropzone Placeholder:

```
// In FileUpload.tsx: renders thumbnail if file exists, else shows Upload icon
{localPreview || value ? (
  <div className="flex items-center gap-3">
    <img src={localPreview || ''} alt="Logo preview" />
  </div>
) : (
  <><Upload size={18} /><span>Upload logo or drag and drop</span></>
)}
```

Conditional Action Disabling:

```
// In ReviewConfirm.tsx:
<button
  disabled={!canCreateAccount}
  className={canCreateAccount ? 'bg-[#0f1330] text-white' : 'bg-[#e9edf5] text-[#98a2b3]'}
>
  Create account
</button>
```

26 Responsive Breakpoint Behavior

26.1 Viewport Breakpoints in the Codebase

The application leverages Tailwind's mobile-first breakpoint system:

Device Class	Breakpoint	Left Branding Panel Behavior	Right Form Panel Behavior
Mobile Phones	<code>< 640px</code>	Completely hidden (<code>hidden</code>). Saves bandwidth and vertical screen space.	Full-width (<code>w-full</code>). Inputs, buttons, and headers stack into a single column. Padding: <code>px-4 pt-8</code> .
Tablets	<code>640px - 1023px</code> (<code>sm:</code>)	Hidden. Focus remains 100% on fast, frictionless onboarding.	Comfortable padded container (<code>px-6 sm:pt-[60px]</code>). 2-column fields (First/Last name, Country/State, City/Timezone) render side-by-side.
Desktop / Laptops	<code>≥ 1024px</code> (<code>lg:</code>)	Visible at 50% width (<code>lg:w-1/2</code>). Fixed height (<code>h-full overflow-hidden</code>). Interactive orbital system active.	Visible at 50% width (<code>lg:w-1/2</code>). Independent scrolling via <code>overflow-y-auto .no-scrollbar</code> . Top padding aligned at <code>lg:pt-[104px]</code> .

27 Error Handling, Guardrails & Lockout Logic

27.1 Form Error Handling Pipeline

When invalid data is detected, the error propagates from the validation function down to the UI:

```
User types "invalid-email" in Work email
|
|   ▼ onBlur event fires
validateEmail("invalid-email") returns "Please enter a valid email address"
|
|   ▼ errors state updated: setErrors({ email: "Please enter a valid email address" })
FormField receives error prop
|
|   └─ Input border turns red: border-red-400 focus:border-red-500
|   └─ Accessible aria-invalid={true} and aria-describedby="work-email-error" applied
|   └─ Error paragraph renders:
```

27.2 The 5-Attempt Security Lockout Algorithm

In `src/pages/EnterPassword.tsx`, enterprise security policy is simulated:

```
const MAX_ATTEMPTS = 5;
const DEMO_PASSWORD = 'password123';

const handleSignIn = () => {
  const trimmed = password.trim();
  if (trimmed === DEMO_PASSWORD) {
    navigate('/2fa');
    return;
  }
  const next = attempts + 1;
  setAttempts(next);
  if (next ≥ MAX_ATTEMPTS) {
    navigate('/locked'); // ← Automatically redirects to /locked!
    return;
  }
  setError(`Incorrect password. You have ${MAX_ATTEMPTS - next} attempts remaining.`);
};
```

28 Backend Integration Status & Mock Architecture

28.1 Explicit Engineering Reality Check

CURRENT IMPLEMENTATION STATUS

This project currently uses frontend-only mock and state simulation. There is currently no active Axios/fetch HTTP client, REST API endpoint, GraphQL server, or backend database actively wired to these forms.

Every data transition (such as passing organization name from Step 1 to Step 2, and generating the workspace subdomain on Step 5) is managed in-memory by React's `OnboardingContext`.

28.2 How to Wire Up the Real Backend in the Future

When the backend team delivers the Java Enterprise Spring Boot microservices, the integration points are clearly isolated:

1. **Step 1 (Check Workspace Availability):** Call `GET /api/v1/workspaces/verify?slug={orgCode}` inside `OrganizationForm.tsx` before proceeding.
2. **Step 2 (Upload Logo to S3/Cloud Storage):** In `FileUpload.tsx`, dispatch a `POST /api/v1/assets/upload` with `FormData`.
3. **Step 3 (Create Tenant Transaction):** In `ReviewConfirm.tsx`, replace `navigate('/welcome')` with:

```
const response = await fetch('/api/v1/onboarding/provision', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ organization, admin })
});
if (response.ok) navigate('/welcome');
```

29 Authentication & Security Simulation

29.1 Simulated Security Capabilities

Workspace–First Tenant Isolation: Enforces multi-tenant SaaS architecture where user identities are scoped under tenant workspaces (e.g. `acmecorp.oneenterprise.io`).

Super Admin Role Assignment: Displays the blue notice badge highlighting that the root account receives the privileged `SUPER_ADMIN` role.

Enterprise Password Complexity Policy: Validates length (8+), numbers (0–9), and symbols (!@#\$\$%) before allowing admin creation.

Two-Factor Authentication (2FA) Simulation: A 6-box OTP entry component with auto-focus advancing and 5-minute countdown simulation.

Brute-Force Lockout Defense: Locks account for 15 minutes after 5 consecutive bad password attempts and redirects to `/Locked`.

30 & 31 React & TypeScript Concepts Checklists

30.1 React Concepts Master Checklist

- ✓ **JSX Syntax:** Used across all components for declarative HTML-in-JS templating.
- ✓ **Functional Components:** 100% of components are modern React functional components with TypeScript typing (`React.FC`).
- ✓ **Props & Prop Drilling Avoidance:** Used for atomic UI controls (`FormField` , `StepProgress`), while React Context eliminates prop drilling for onboarding data.
- ✓ **useState Hook:** Controls form inputs, error objects, visibility toggles, and tab switches.
- ✓ **useCallback Hook:** Memoizes `updateOrganization` , `updateAdmin` , and `resetOnboarding` inside `OnboardingContext.tsx` to prevent unnecessary child re-renders.
- ✓ **useRef Hook:** Manages DOM focus in `TwoFactor.tsx` (input refs array), `FileUpload.tsx` (file input ref), and `OrganizationForm.tsx` (org-code manual edit ref).
- ✓ **useEffect Hook:** Resets vertical window scroll position to `scrollTop = 0` upon page transitions.
- ✓ **useContext & Custom Hook:** `useOnboarding()` provides centralized access to the onboarding context store.
- ✓ **Conditional Rendering:** Implemented via `&&` , ternaries, and dynamic CSS classes.
- ✓ **Client-Side Routing:** Integrated via `react-router-dom` with nested layout routes and `<Outlet />` .

31.1 TypeScript Concepts Master Checklist

- ✓ **Interfaces:** `OrganizationDetails` , `AdminAccount` , `OnboardingState` , `FormFieldProps` , `StepProgressProps` .
- ✓ **Type Aliases:** `SocialProvider = 'google' | 'microsoft' | 'sso'` in `SocialLoginButton.tsx` .
- ✓ **Union Types:** Tab state in `VisualBrandingPanel` : `activeTab: 'secure' | 'scalable' | 'future'` .
- ✓ **Utility Types (`Partial<T>`):** Used in context updater methods for partial field mutations.
- ✓ **HTML Element Prop Inheritance:** `FormFieldProps` extends `InputHTMLAttributes<HTMLInputElement | HTMLSelectElement>` .

32 Important Line-by-Line Code Walkthroughs

32.1 Code Walkthrough 1: Application Bootstrap (`src/main.tsx`)

```
1: import { StrictMode } from 'react'
2: import { createRoot } from 'react-dom/client'
3: import './index.css'
4: import App from './App.tsx'
5:
6: createRoot(document.getElementById('root')!).render(
7:   <StrictMode>
8:     <App />
9:   </StrictMode>,
10: )
```

Line 2: Imports `createRoot`, the modern React 19 concurrent renderer entry point.

Line 3: Imports `./index.css`, injecting Tailwind v4 and Google Fonts globally before components render.

Line 6: `document.getElementById('root')!` locates the HTML root div. The TypeScript non-null assertion operator (`!`) asserts that the element definitely exists in the DOM.

Lines 7–9: Wraps `<App />` in `<StrictMode>` to activate dev-mode double-rendering checks.

32.2 Code Walkthrough 2: Custom Hook Guard (`src/context/useOnboarding.ts`)

```
1: import { useContext } from 'react';
2: import { OnboardingContext } from './OnboardingContextValue';
3: import type { OnboardingContextType } from './OnboardingContextValue';
4:
5: export const useOnboarding = (): OnboardingContextType => {
6:   const context = useContext(OnboardingContext);
7:   if (!context) {
8:     throw new Error('useOnboarding must be used within an OnboardingProvider');
9:   }
10:  return context;
11: };
```

Lines 5–6: Consumes `OnboardingContext` via native `useContext`.

Lines 7–9: Fail-Fast Defensive Guard. If a developer accidentally renders a component using `useOnboarding()` outside the `<OnboardingProvider>` tree, React immediately throws a clear, actionable runtime error rather than silently failing with `TypeError: Cannot read properties of undefined!`

33 How to Explain This Project in a POC or Interview

33.1 Senior Developer Speaking Script

DELIVER THIS EXPLANATION WITH CONFIDENCE

"Good morning / afternoon everyone. I'm excited to walk you through **SignIn-UI-Frontend**, our modern enterprise authentication and workspace onboarding platform for One Enterprise.

Our primary business goal was to solve a major friction point in B2B SaaS: enterprise onboarding abandonment. Rather than overwhelming administrators with a single massive 25-field form, we engineered a modular 3-step progressive onboarding pipeline.

On screens 1024px and wider, we implement a **50/50 Split Screen Architecture** using our `AuthLayout`. The left side is a persistent, non-scrolling branding panel that highlights our enterprise trust certifications—SOC 2 Type II, ISO 27001, and our 99.95% uptime SLA—along with an interactive visual orbital diagram illustrating how *People*, *Process*, and *Progress* interconnect through One Enterprise.

The right side hosts our scrollable form views powered by React Router v7. In Step 1, the user enters their organization details. Our code auto-generates a clean workspace slug in real time and dynamically populates regional states based on country selection. When they click Continue, this data is preserved in our centralized React Context store. Step 2 collects root Super Administrator credentials with enforced password complexity. Step 3 acts as our legal compliance gateway, strictly requiring confirmation of authority and Data Processing Agreements before the tenant is provisioned.

Technologically, we built this on React 19, TypeScript, Vite, Tailwind CSS v4, and Material UI v9. Everything is fully typed, accessible, and responsive. I'd be glad to dive into any specific component or architectural layer!"

33.2 Likely Interview / POC Questions & Model Answers

Q1: Why did you use React Context instead of Redux or Zustand?

Answer: "For this specific multi-step onboarding wizard, Redux would have introduced unnecessary boilerplate and bundle weight. Our onboarding workflow has a single, cohesive state domain: `organization` and `admin` data. React's native Context API combined with `useCallback` memoization gave us 100% of the cross-step state persistence we needed with zero external dependencies and minimal bundle footprint."

Q2: How does the application prevent losing data when a user clicks 'Back'?

Answer: "When a user fills Step 1 and clicks Continue, the handler dispatches `updateOrganization(...)` to our `OnboardingContext`. When the user is on Step 2 (Admin Account) and clicks 'Back', the router navigates back to `/organization`. When `OrganizationForm` mounts, its local `useState` hooks are initialized directly from `organization.*` in Context. As a result, all previously selected inputs, dropdowns, and uploaded files are seamlessly restored."

Q3: How does the 50/50 split layout behave on mobile devices?

Answer: "In `AuthLayout.tsx`, the left branding container is styled with `hidden lg:block lg:w-1/2`. On viewports below 1024px (tablets and phones), the left branding panel is completely removed from layout calculation. The right form container automatically expands to `w-full`, delivering a clean, single-column mobile experience with touch-friendly input targets."

34.1 The 13-Step Development Story

1. **Step 1 (Scaffolding):** Initialized Vite with React 19 and TypeScript template.
2. **Step 2 (Tooling):** Installed Tailwind CSS v4 via `@tailwindcss/vite`, Material UI v9, and Lucide icons.
3. **Step 3 (Global Styles):** Configured `index.css` with Google Fonts (Inter, JetBrains Mono) and custom `@theme` color tokens.
4. **Step 4 (Type Contracts):** Authored `src/types/onboarding.ts` to define strict TypeScript interfaces for all organization and admin data models.
5. **Step 5 (Validation Engine):** Built `src/utils/validation.ts` with unit-level validation functions and auto-slug generation.
6. **Step 6 (Context Store):** Implemented `OnboardingContext.tsx` with `useCallback` memoized updaters and the `useOnboarding` custom hook.
7. **Step 7 (Atomic UI Controls):** Built reusable primitives: `FormField`, `FileUpload`, `PrimaryButton`, `StepProgress`, and `AgreementCheckbox`.
8. **Step 8 (Visual Branding Stage):** Engineered `VisualBrandingPanel.tsx` featuring full mountain image background, SVG orbital system, glassmorphic 1E card, and interactive tabs.
9. **Step 9 (AuthLayout Master):** Created `AuthLayout.tsx` establishing the 50/50 split desktop layout with `<Outlet />`.
10. **Step 10 (Onboarding Flow Pages):** Assembled `OrganizationForm.tsx`, `AdminAccountPage.tsx`, `ReviewConfirm.tsx`, and `Welcome.tsx`.
11. **Step 11 (Secondary Auth Suite):** Integrated login views: `EnterPassword.tsx` (with 5-attempt lockout counter), `TwoFactor.tsx` (with 6-box auto-advancing OTP), `ForgotPassword.tsx`, and `AccountLocked.tsx`.
12. **Step 12 (Routing Matrix):** Configured `src/App.tsx` with React Router v7 routes, redirects, and fallbacks.
13. **Step 13 (Refinement & Linting):** Executed `oxlint` and TypeScript compilation checks (`tsc -b`) to ensure zero errors and clean bundle output.

35.1 Engineering Decisions: "Why Was It Written This Way?"

Why Tailwind v4 instead of CSS Modules? Tailwind v4 provides rapid utility styling with zero CSS runtime overhead and built-in CSS variable theming.

Why Controlled Inputs instead of Uncontrolled refs? Controlled components enable real-time field validation, dynamic slug auto-generation on keystroke, and conditional button enabling.

Why Native SVG instead of Canvas for Orbital Diagram? SVG vectors scale infinitely across Retina and 4K displays without blurring, support CSS animations, and have zero impact on JavaScript main thread execution.

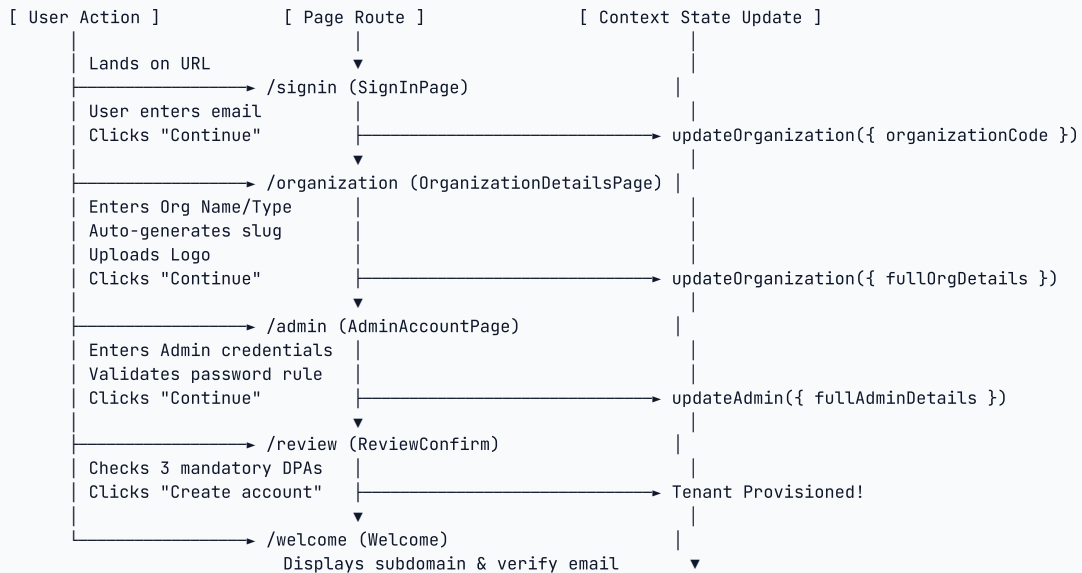
36 Technical Improvement Roadmap

In technical discussions, demonstrating awareness of future architectural enhancements proves seniority. Here is the clear division between current implementation and future roadmap:

Domain	Current Implementation	Possible Future Enhancement
Data Persistence	In-memory React Context (data resets on hard browser page refresh).	Persist draft onboarding state to <code>sessionStorage</code> or <code>localStorage</code> so users can refresh without losing progress.
Schema Validation	Custom imperative functions in <code>validation.ts</code> .	Migrate to a declarative schema validation library like Zod or Yup integrated with React Hook Form.
Backend API	Frontend mock state simulation.	Implement an Axios or TanStack Query (React Query) layer with retry logic, loading spinners, and error boundary wrappers.
Automated Testing	TypeScript static type analysis and oxlint linting.	Add Vitest and React Testing Library test suites covering form validation and routing transitions.
Internationalization	Hardcoded English labels and messages.	Extract UI strings into an <code>i18next</code> translation dictionary for multi-language global enterprise support.

37 Complete System Architecture Diagrams

37.1 Multi-Step Onboarding State & Navigation Sequence



38 Technical Project Glossary

Term	Simple Definition	How it Connects to SignIn-UI-Frontend
Single-Page App (SPA)	A web app that rewrites the current web page with new data from the web server, instead of the default method of a browser loading entire new pages.	Our application runs as an SPA using React Router v7. Navigation between <code>/signin</code> and <code>/organization</code> happens instantly in-memory.
Controlled Component	An input element whose value is controlled by React state.	All inputs in <code>OrganizationForm</code> and <code>AdminAccountPage</code> bind their <code>value</code> and <code>onChange</code> to React state.
React Context	A React feature for sharing values between components without having to explicitly pass a prop through every level of the tree.	<code>OnboardingContext</code> stores company profile and admin information across all onboarding screens.
Layout Route	A parent route in React Router that renders a common layout containing an <code><Outlet /></code> for child routes.	<code>AuthLayout.tsx</code> serves as the layout route, rendering the sticky left branding panel once for all routes.
Tailwind Utility Class	Single-purpose CSS classes that apply specific styling rules directly in JSX.	Classes like <code>flex</code> , <code>h-screen</code> , <code>bg-[#0f1330]</code> , and <code>rounded-[28px]</code> compose the entire design.
Glassmorphism	A modern UI trend using translucent backgrounds with background blur filters to create a frosted glass look.	Used in <code>VisualBrandingPanel.tsx</code> for the center IE card (<code>backdrop-blur-xl bg-white/90</code>).
Fail-Safe Guard	Code designed to detect improper usage and fail early with an informative diagnostic error.	<code>useOnboarding.ts</code> checks <code>if (!context) throw new Error(...)</code> .

39 Final Project Cheat Sheet

SIGNIN-UI-FRONTEND QUICK CHEAT SHEET

Project Name: SignIn-UI-Frontend (package: stackly-auth)
Core Technologies: React 19.2, TypeScript 6.0, Vite 8.3, Tailwind CSS v4, MUI v9
Entry Point: src/main.tsx (mounts into #root via createRoot)
Root Router: src/App.tsx (BrowserRouter, Routes, Route, Navigate)
Layout Container: src/components/AuthLayout.tsx (50/50 Desktop Split Layout)
Left Branding Panel: src/components/VisualBrandingPanel.tsx (Mountains, SVG Orbital, 1E Core)
State Management: src/context/OnboardingContext.tsx (Context API + useOnboarding hook)
Validation Engine: src/utils/validation.ts (validateRequired, validateEmail, validatePassword)

PRIMARY ONBOARDING ROUTES:

1. /signin → SignInPage.tsx (Workspace & Work email identification)
2. /organization → OrganizationDetailsPage.tsx / OrganizationForm.tsx (Company info)
3. /admin → AdminAccountPage.tsx (Super Admin credentials & password rule)
4. /review → ReviewConfirm.tsx (Mandatory Terms of Service, Authority, DPA)
5. /welcome → Welcome.tsx (Provisioned subdomain & email verification)

SECONDARY AUTHENTICATION ROUTES:

- /login → EnterPassword.tsx (Demo pass: password123, 5-attempt lockout)
- /2fa → TwoFactor.tsx (6-digit auto-advancing OTP verification)
- /forgot-password → ForgotPassword.tsx (Self-service recovery)
- /check-email → CheckEmail.tsx (Reset link dispatch notice)
- /locked → AccountLocked.tsx (15-minute brute-force lockout screen)
- /success → Success.tsx (Post-login dashboard redirect screen)

40 Self-Assessment & Mastery Checklist

40.1 Project Knowledge Verification Checklist

- ☒ **Architecture:** I can explain the 50/50 split layout in `AuthLayout.tsx` and why it prevents layout shift.
- ☒ **Startup Sequence:** I can trace execution from `index.html` to `main.tsx`, `createRoot`, `StrictMode`, and `App.tsx`.
- ☒ **Routing:** I can explain how React Router v7 routes work and how parent layout routes wrap child views via `<Outlet />`.
- ☒ **State Management:** I can explain why React Context was chosen over Redux and how data persists across steps.
- ☒ **Form Controls:** I can explain how controlled inputs work with `value` and `onChange` handlers.
- ☒ **Validation:** I can explain how `validation.ts` checks emails, workspaces, password complexity, and auto-generates slugs.
- ☒ **Styling Stack:** I can explain how Tailwind CSS v4's `@theme` and utility classes cooperate with Material UI v9 components.
- ☒ **Left Branding Panel:** I can describe how the SVG orbital curve paths connect People, Process, and Progress to the 1E center core.
- ☒ **Backend Status:** I can clearly state that this project currently utilizes frontend-only mock state simulation and explain how real APIs will be connected.
- ☒ **POC Confidence:** I can deliver the Senior Developer speaking script and answer technical interview questions without hesitation!